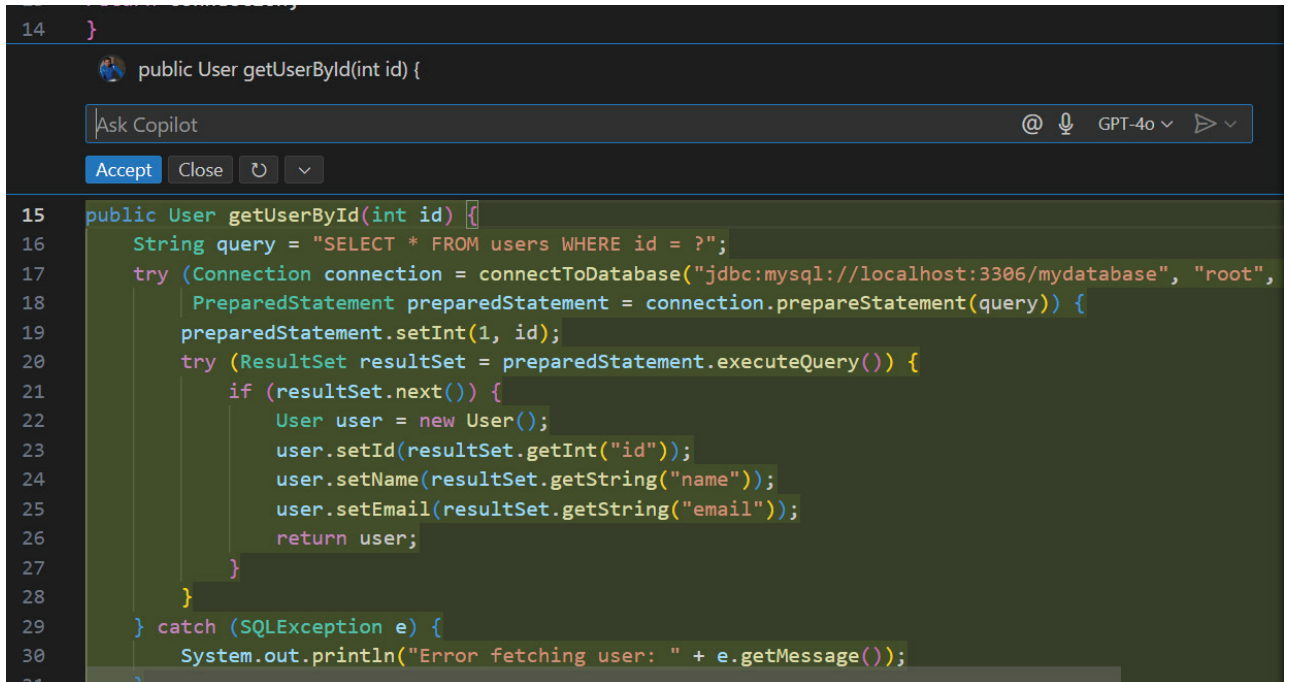


Using natural language in code: One striking feature of GitHub Copilot is its ability to understand and interpret natural language comments. By writing a comment such as “create a method to fetch a user by ID”, Copilot generates the corresponding Java code as shown in Figure 2.

Copilot actions: Copilot can also perform specific actions based on natural language instructions. For example, instructing Copilot to “create a REST endpoint for user retrieval” yields a complete method with the necessary annotations as seen in Figure 3.



The screenshot shows a code editor with a Copilot suggestion. The suggestion is for a method named `getUserById` that takes an `int id` and returns a `User` object. The code uses JDBC to connect to a MySQL database, execute a query, and return the first result. It includes error handling for `SQLException`.

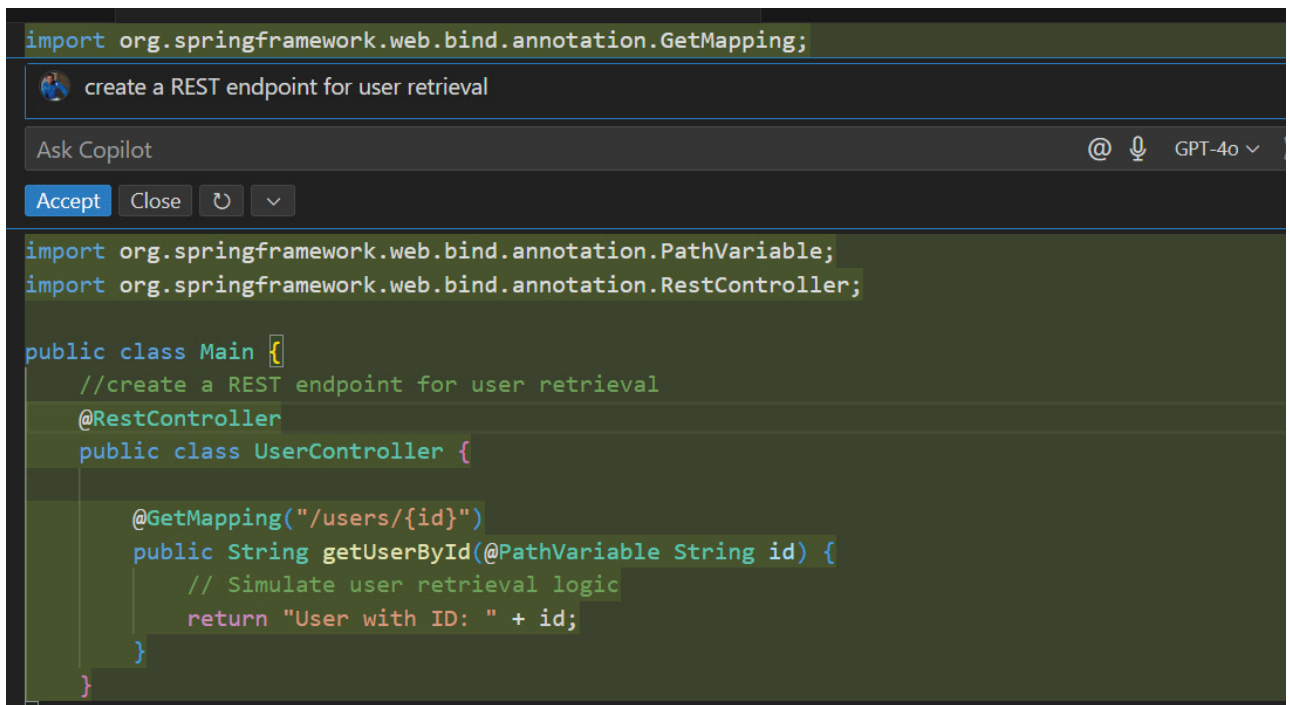
```

14 }

15 public User getUserById(int id) {
16     String query = "SELECT * FROM users WHERE id = ?";
17     try (Connection connection = connectToDatabase("jdbc:mysql://localhost:3306/mydatabase", "root",
18         PreparedStatement preparedStatement = connection.prepareStatement(query)) {
19         preparedStatement.setInt(1, id);
20         try (ResultSet resultSet = preparedStatement.executeQuery()) {
21             if (resultSet.next()) {
22                 User user = new User();
23                 user.setId(resultSet.getInt("id"));
24                 user.setName(resultSet.getString("name"));
25                 user.setEmail(resultSet.getString("email"));
26                 return user;
27             }
28         }
29     } catch (SQLException e) {
30         System.out.println("Error fetching user: " + e.getMessage());
31     }
32 }

```

Figure 2: Copilot generated codes



The screenshot shows a code editor with a Copilot suggestion. The suggestion is for a REST endpoint for user retrieval. The code uses Spring Framework annotations like `@GetMapping` and `@PathVariable` to create a `UserController` with a `getUserById` method. The method simulates user retrieval logic by returning a string.

```

import org.springframework.web.bind.annotation.GetMapping;

create a REST endpoint for user retrieval

import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

public class Main {
    //create a REST endpoint for user retrieval
    @RestController
    public class UserController {
        @GetMapping("/users/{id}")
        public String getUserById(@PathVariable String id) {
            // Simulate user retrieval logic
            return "User with ID: " + id;
        }
    }
}

```

Figure 3: Copilot creating a REST endpoint